

PROJECT BATTLE IQ

เว็บเกมตอบคำถามวัดไอคิวแบบแข่งขัน พัฒนาด้วย Spring Boot
รายวิชา CP353002 Principles of Software Design

สมาชิก

กลุ่มของเรา

5 คน แบ่งงานตามโมดูล แต่ละคนรับผิดชอบ Pattern และหน้าเว็บ
ของตัวเอง

รัฐภูมิ เกิดพระจีน 673380057-5
ระบบเล่นเกม, Factory, Docker

พีรพัฒน์ ป้องกันยา 673380053-3
ผู้ใช้/โปรไฟล์, Observer, ห้องแข่ง

ก้องภพ โชควิริยะ 673380030-5
คำถาม/คำตอบ, Command, Swagger

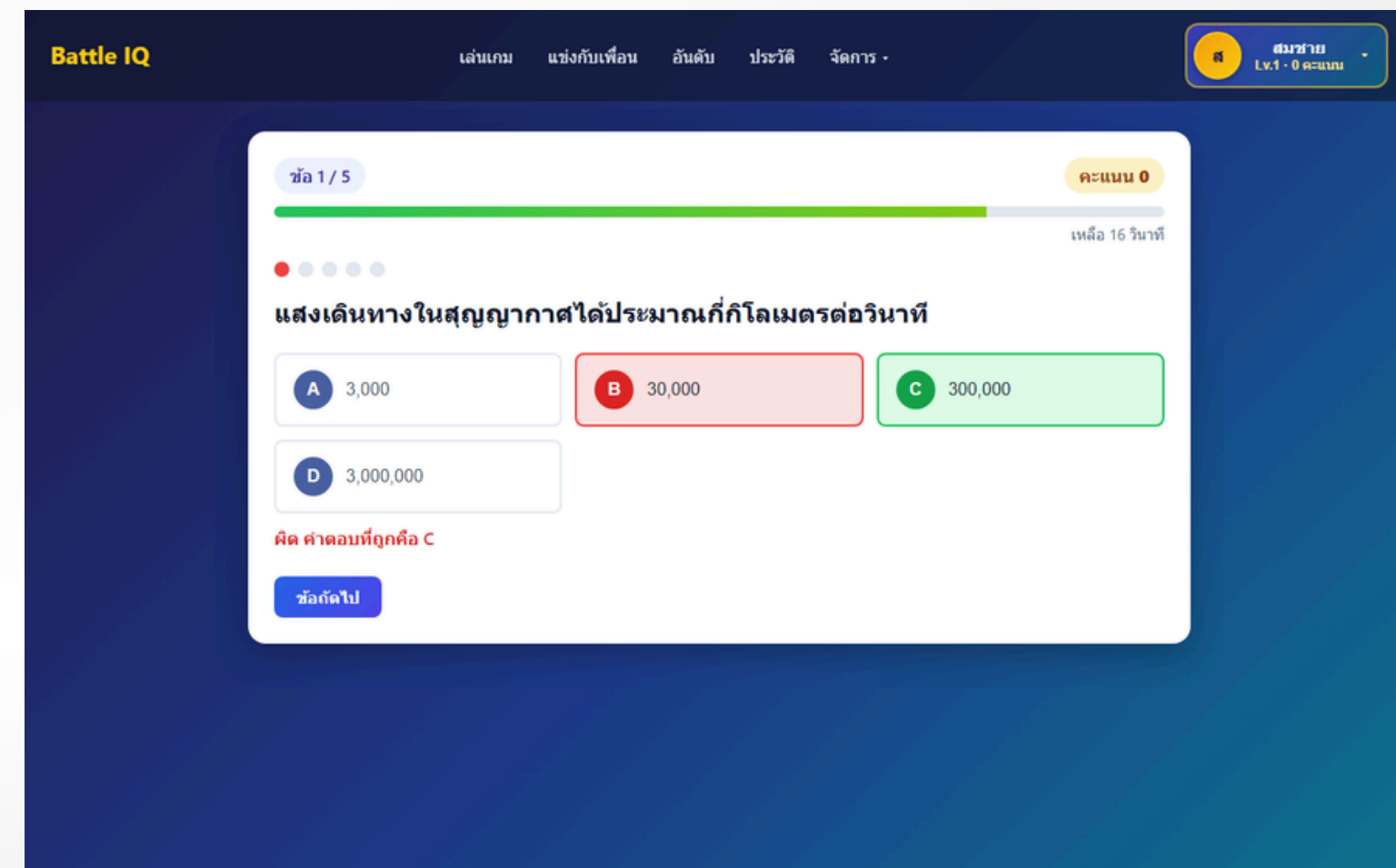
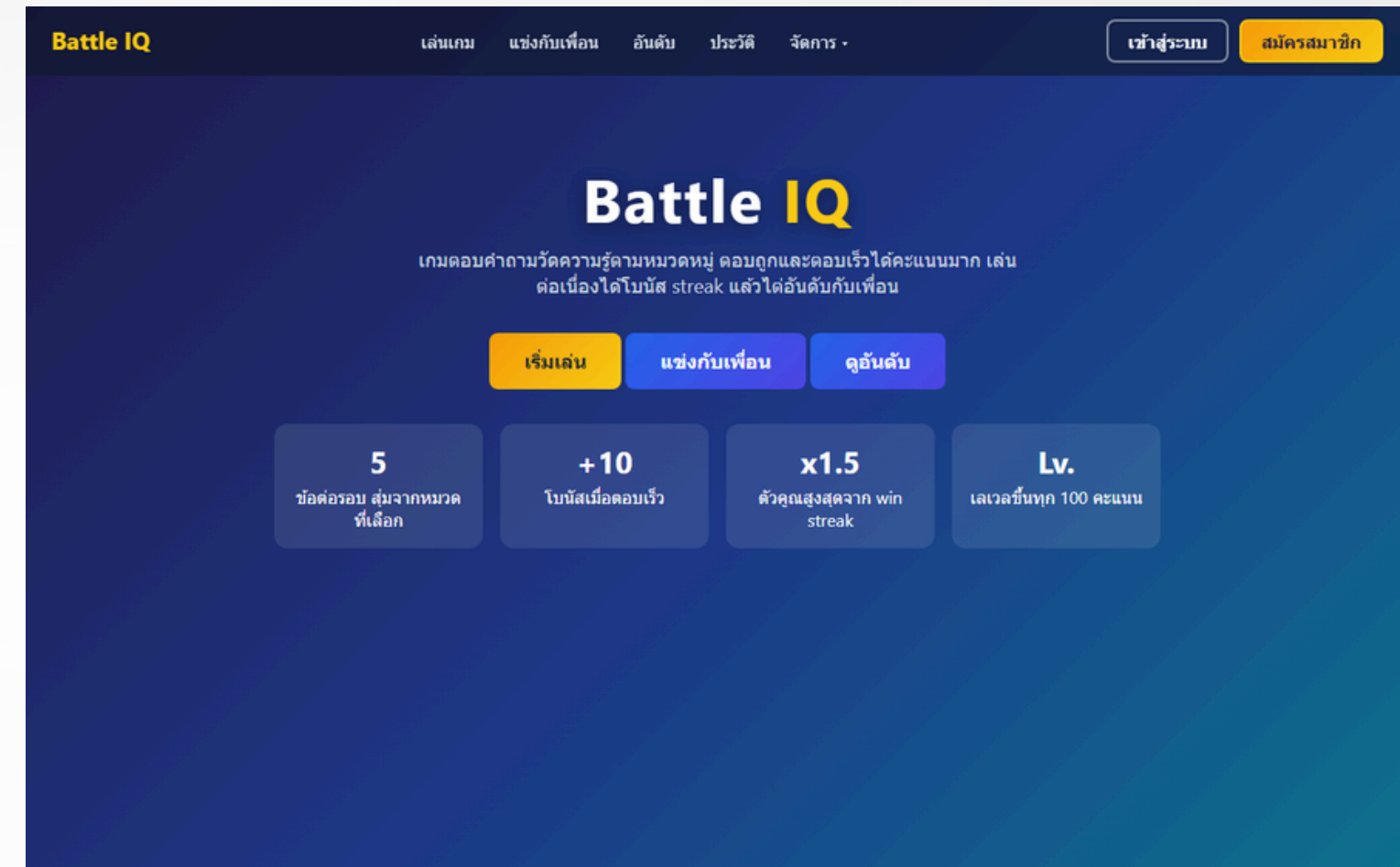
ธนกฤต ละครสวา 673380269-0
คิดคะแนน/ผลการเล่น, Strategy

ณพวิทย์ วงษ์ประเสริฐ 673380266-6
หมวดหมู่, Leaderboard, Deploy Render

ที่มาและวัตถุประสงค์

ผู้เรียนอยากฝึกความรู้รอบตัวแบบสนุก
และแข่งขันได้

Battle IQ จึงเป็นเว็บเกมที่เล่นคนเดียวหรือสร้าง
ห้องแข่งกับเพื่อนได้ มีคะแนน เลเวล สตริก และ
กระดานผู้นำ พร้อมฝึกออกแบบซอฟต์แวร์ตามหลัก
SOLID และ Design Pattern



ฟีเจอร์หลัก 3 กลุ่ม

1

เลือกหมวดหมู่ สุ่มคำถาม จับเวลาต่อข้อ ได้คะแนนพื้นฐาน + โบนัสความเร็ว + ตัวคูณสตรีก เห็นเฉลยทันทีหลังตอบ

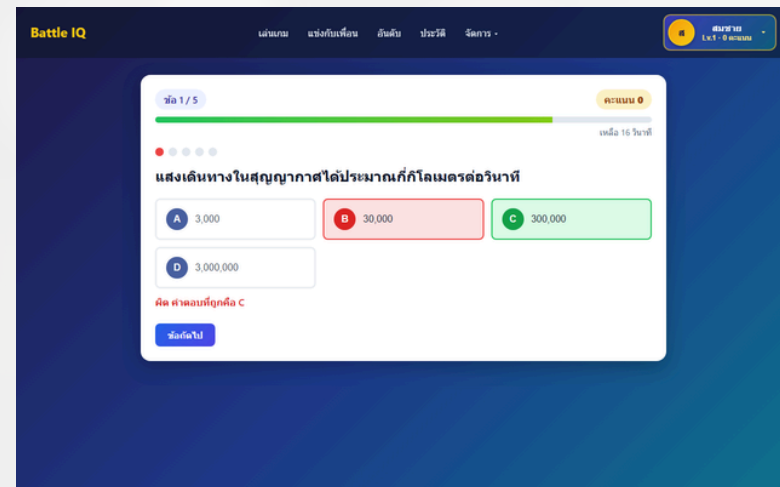
2

สร้างห้องด้วยรหัส 6 หลัก เพื่อนเข้าร่วม ทุกคนได้ชุดคำถามเดียวกัน จบเกมแล้วสรุปอันดับในห้องทันที

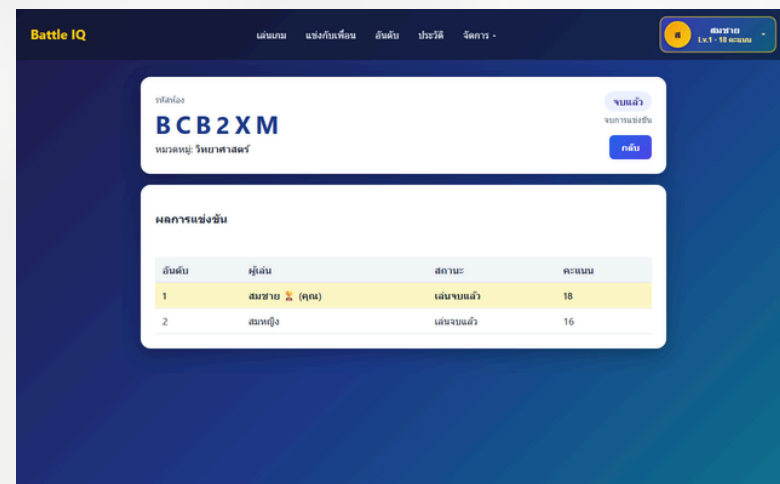
3

สะสมคะแนนรวม เลเวล และสตรีก ดูประวัติการเล่นย้อนหลัง และจัดอันดับผู้เล่นทั้งระบบแบบแบ่งหน้า

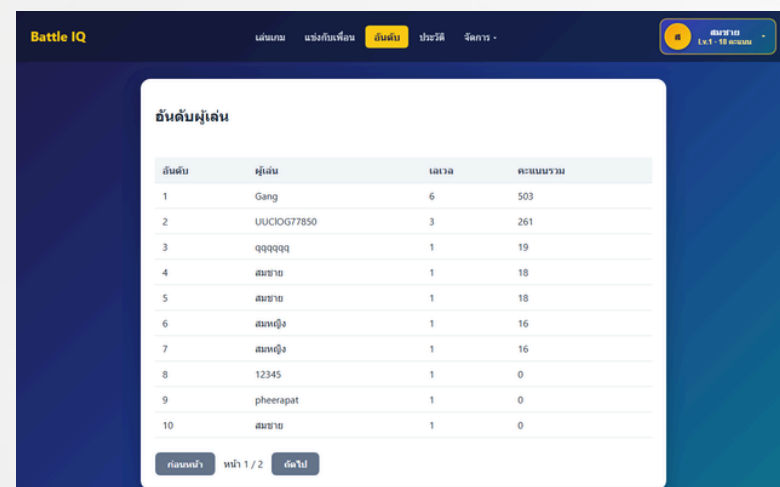
เล่นคนเดียว
(Solo Quiz)



แข่งหลายคน
(Multiplayer Room)



โปรไฟล์และ
Leaderboard



BATTLE IQ

ในตัวเลข

ขนาดของระบบที่พัฒนา ทดสอบ และ deploy ใช้งานจริงแล้ว

22

REST API endpoints
พร้อม Swagger UI

8

ตารางฐานข้อมูล
PostgreSQL บน Neon

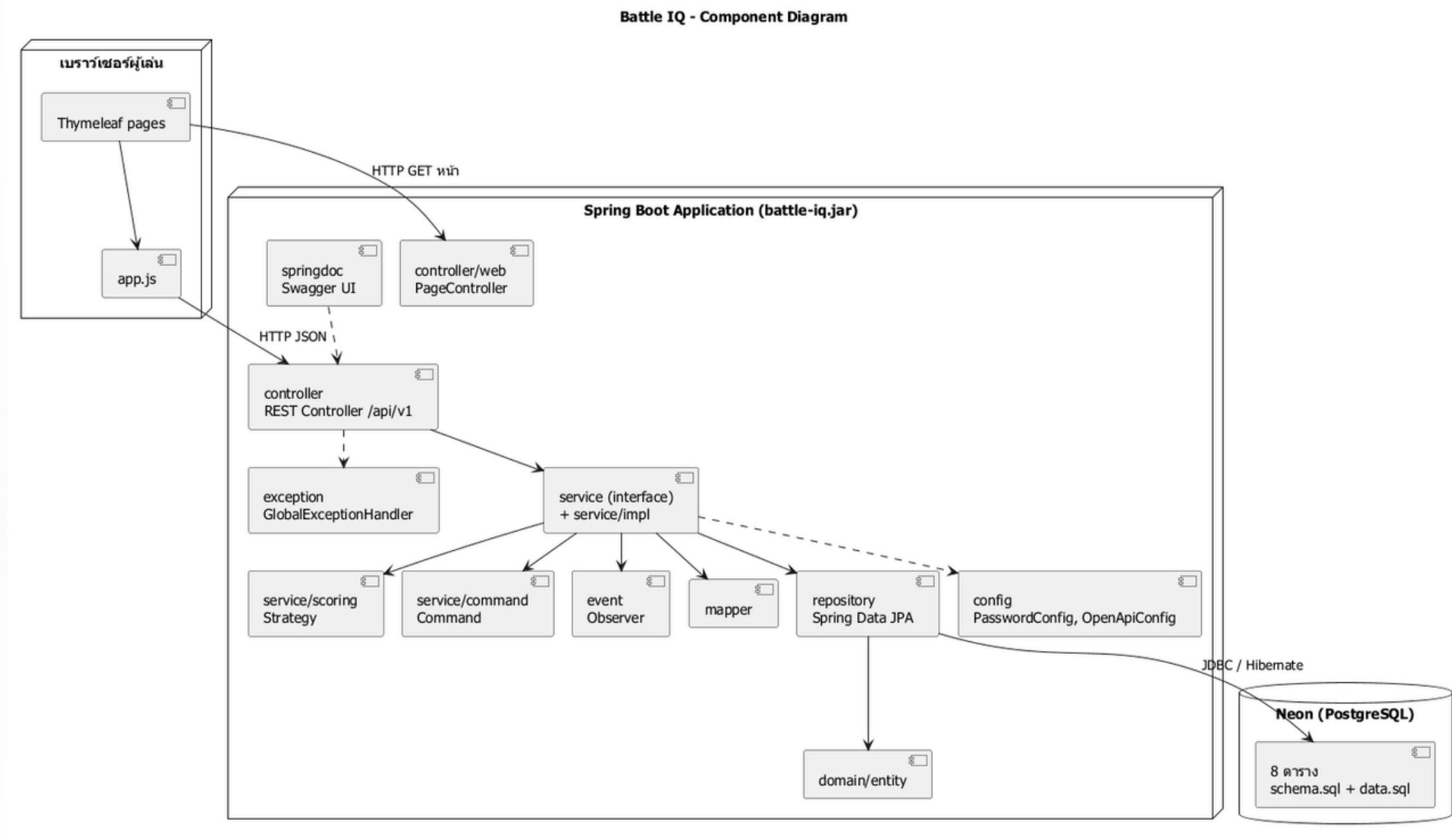
54

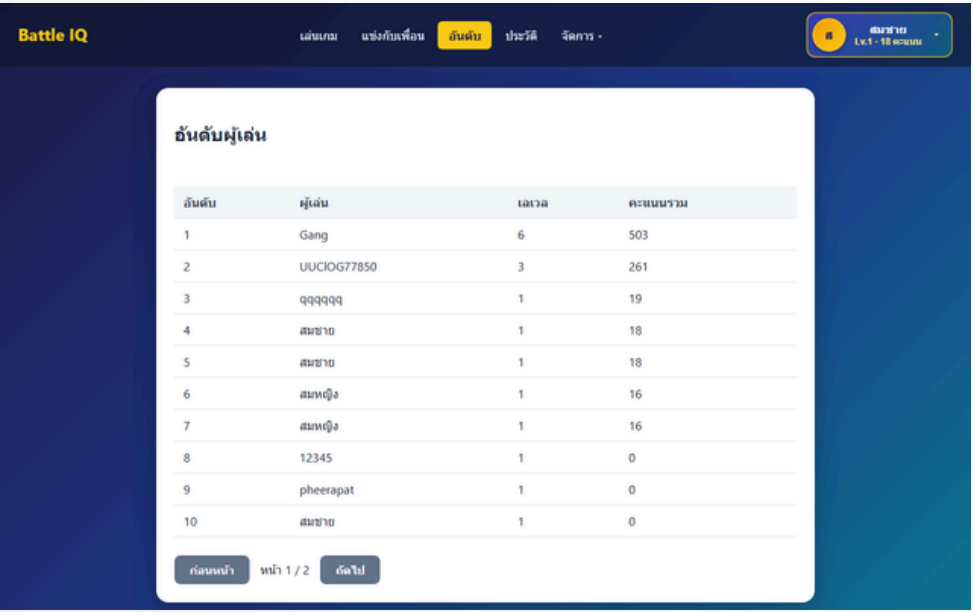
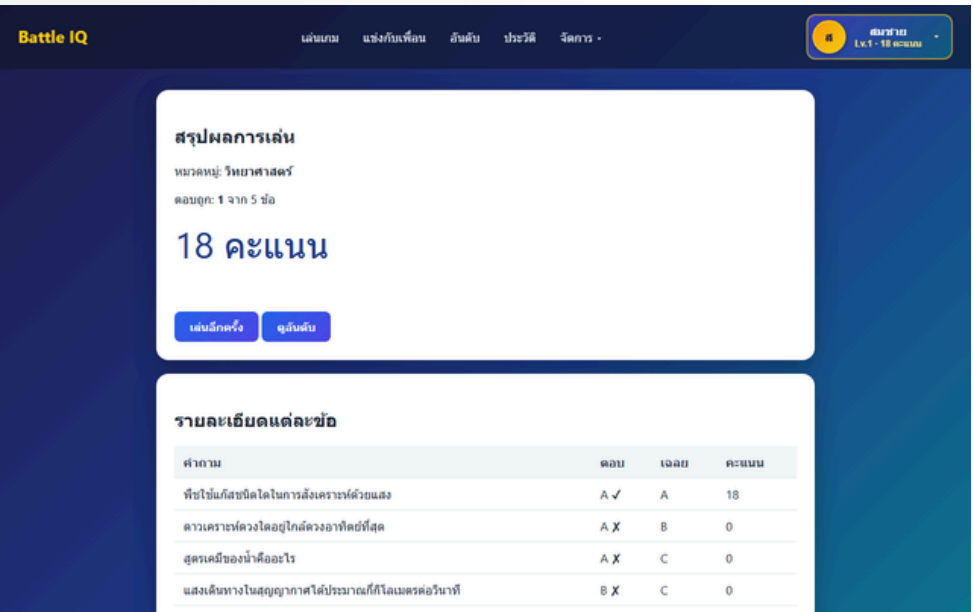
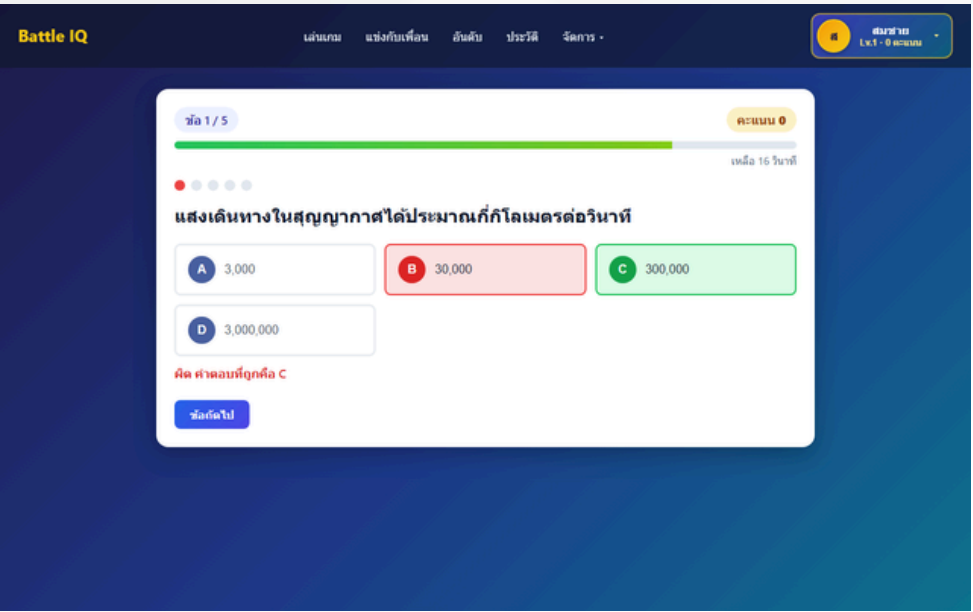
เทสผ่านทั้งหมด
Unit + Integration

สถาปัตยกรรม LAYERED

Controller → Service (interface + impl) → Repository → Entity แยกชั้นชัดเจน ใช้ DTO และ Mapper กันไม่ให้ Entity หลุดออกไปหาผู้ใช้

Java 17, Spring Boot 4, Spring Data JPA, Thymeleaf + JavaScript, PostgreSQL (Neon), Docker uu Render และ CI/CD ด้วย GitHub Actions





DESIGN PATTERNS

Strategy – ScoringStrategy (Basic / TimeBonus / Streak) เปลี่ยนวิธีคิดคะแนนได้โดยไม่แก้ไขโค้ดเดิม

Observer – GameCompletedEvent อัปเดตโปรไฟล์หลังจบเกมแบบไม่ผูกติดกัน

Command – SubmitAnswerCommand ห่อการตอบคำถามให้ทดสอบและนำกลับมาใช้ได้

Factory – QuestionFactory สร้างคำถามจาก DTO ที่จุดเดียว

หลักการ SOLID

S – แต่ละ Service รับผิดชอบงานเดียว เช่น RoomService ดูแลเฉพาะห้องแข่ง
O – เพิ่มกลยุทธ์คะแนนใหม่ด้วยการเพิ่มคลาส ไม่แก้ไขของเดิม

L – ทุก ScoringStrategy ใช้แทนกันได้ผ่าน interface เดียว

I – แยก interface ของ Service ตามหน้าที่ ไม่บังคับ implement เกินจำเป็น

D – Controller พึ่งพา interface ของ Service และรับผ่าน constructor injection

ผลลัพธ์: โค้ดทดสอบง่ายด้วย Mockito
และขยายฟีเจอร์ได้โดยไม่กระทบส่วนอื่น

Battle IQ

เล่นเกมแข่งกับเพื่อนอันดับประวัติจัดการ

ส. สมชาย Lv.1 · 18 คะแนน

รหัสห้อง

BCB2XM

หมวดหมู่: วิทยาศาสตร์

จบแล้ว

จบการแข่งขัน

กลับ

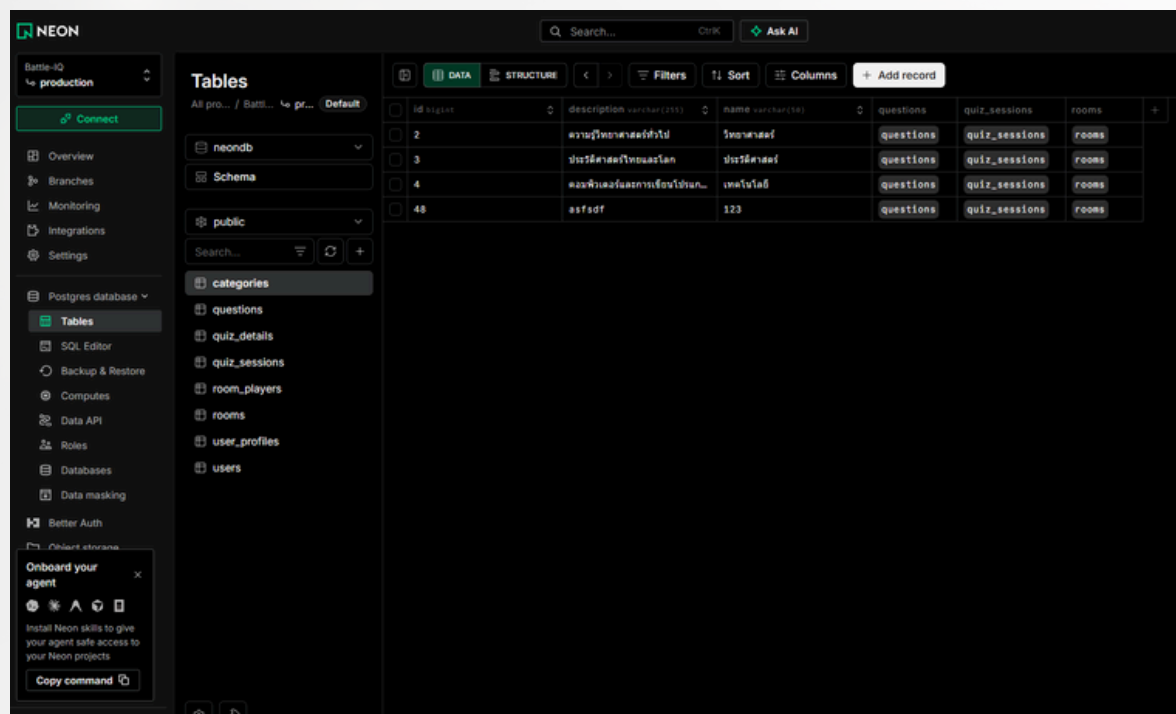
ผลการแข่งขัน

อันดับ	ผู้เล่น	สถานะ	คะแนน
1	สมชาย 🏆 (คุณ)	เล่นจบแล้ว	18
2	สมหญิง	เล่นจบแล้ว	16

ฐานข้อมูล POSTGRESQL

ใช้ PostgreSQL บน Neon (cloud) ทั้งทีมเชื่อม
ฐานข้อมูลเดียวกันผ่านไฟล์ .env ไม่ต้องติดตั้งใน
เครื่อง

8 ตาราง: users, user_profiles, categories,
questions, quiz_sessions, quiz_details,
rooms, room_players



The screenshot shows the Neon database interface. On the left, there's a sidebar with navigation options like Overview, Branches, Monitoring, Integrations, Settings, and Postgres database. The main area displays a table structure for a database named 'production'. The table has columns: id (integer), description (varchar(255)), name (varchar(255)), questions, quiz_sessions, and rooms. There are four rows of data, each with a checkbox in the first column.

	id (integer)	description (varchar(255))	name (varchar(255))	questions	quiz_sessions	rooms
☐	2	ความรู้เกี่ยวกับเทคโนโลยี	เทคโนโลยี	questions	quiz_sessions	rooms
☐	3	ประวัติศาสตร์และเทคโนโลยี	ประวัติศาสตร์	questions	quiz_sessions	rooms
☐	4	ความรู้เกี่ยวกับเทคโนโลยีและ...	เทคโนโลยี	questions	quiz_sessions	rooms
☐	48	asdf	123	questions	quiz_sessions	rooms

1

ออกแบบ ER Diagram ด้วย draw.ioระบุ PK /
FK / UK ครบทุกตาราง พร้อม Data Dictionary

2

schema.sql + data.sql สร้างตารางและข้อมูล
ตัวอย่าง (หมวดหมู่ คำถาม ผู้ใช้ demo)
อัตโนมัติตอนรัน

3

Hibernate (JPA) แมป Entity กับตาราง รองรับ
ความสัมพันธ์ 1-1, 1-N และ M-N (users ↔
rooms)

การทดสอบ ตามหลัก SQA

วางแผนทดสอบตามหลัก SQA: Unit Test (JUnit 5 + Mockito) และ Integration Test (Spring Boot Test + MockMvc) รวม 54 เทส ผ่านทั้งหมด

ทดสอบผ่านหน้าเว็บจริง 9 กรณี (TC-M01 ถึง M09) และทดสอบ validation ของ API เช่น ส่ง username ยาว 500 ตัวอักษร ต้องได้ 400 ไม่ใช่ 500



1

Unit Test 32 เทส: Service, Scoring Strategy, Command, Room

2

Integration Test 21 เทส: User API และ API Validation ทุกเทส rollback

3

รันอัตโนมัติทุก push ด้วย GitHub Actions พร้อมรายงาน Surefire ใน test/report

การ DEPLOY และ CI/CD

Docker multi-stage build รันบน Render
(Singapore, free tier) ใช้งานได้จริงที่ battle-
iq.onrender.com

GitHub Actions: ทุก push เข้า develop
จะ build → test → เรียก Render Deploy Hook
ให้ deploy อัตโนมัติ

RENDER
+
DOCKER

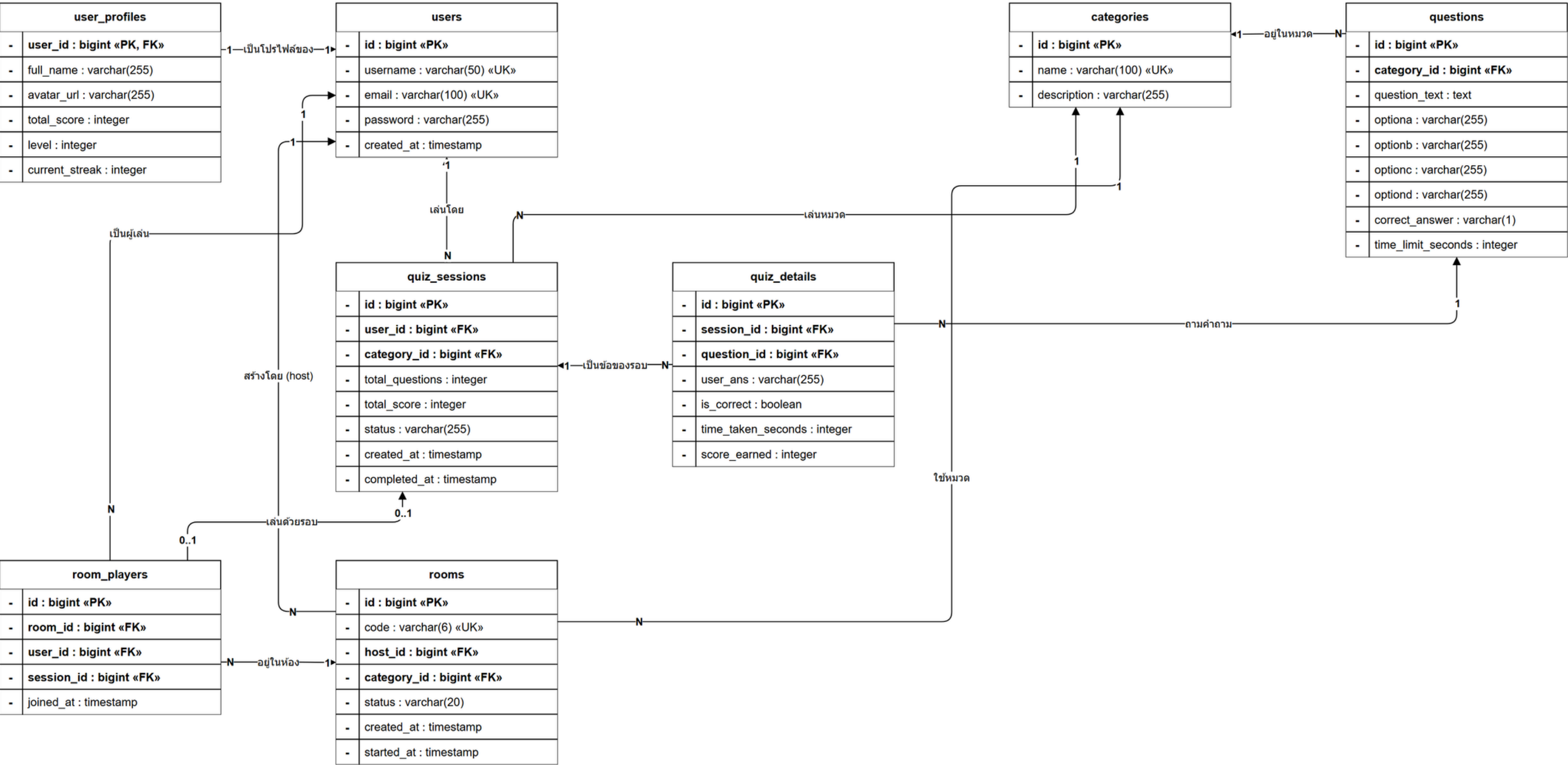
เว็บรันใน container เดียว ตั้ง
JAVA_TOOL_OPTIONS=-Xmx400m
ให้พอดีกับ RAM 512 MB ของ free tier

GITHUB
ACTIONS
CI/CD

ทีมส่งงานผ่าน Pull Request เข้า
develop, CI ตรวจ build/test ก่อน
merge แล้ว deploy ให้เอง

ER DIAGRAM

Battle IQ - ER Diagram (PostgreSQL, 8 ตาราง)



DEMO



[Battle IQ](#)

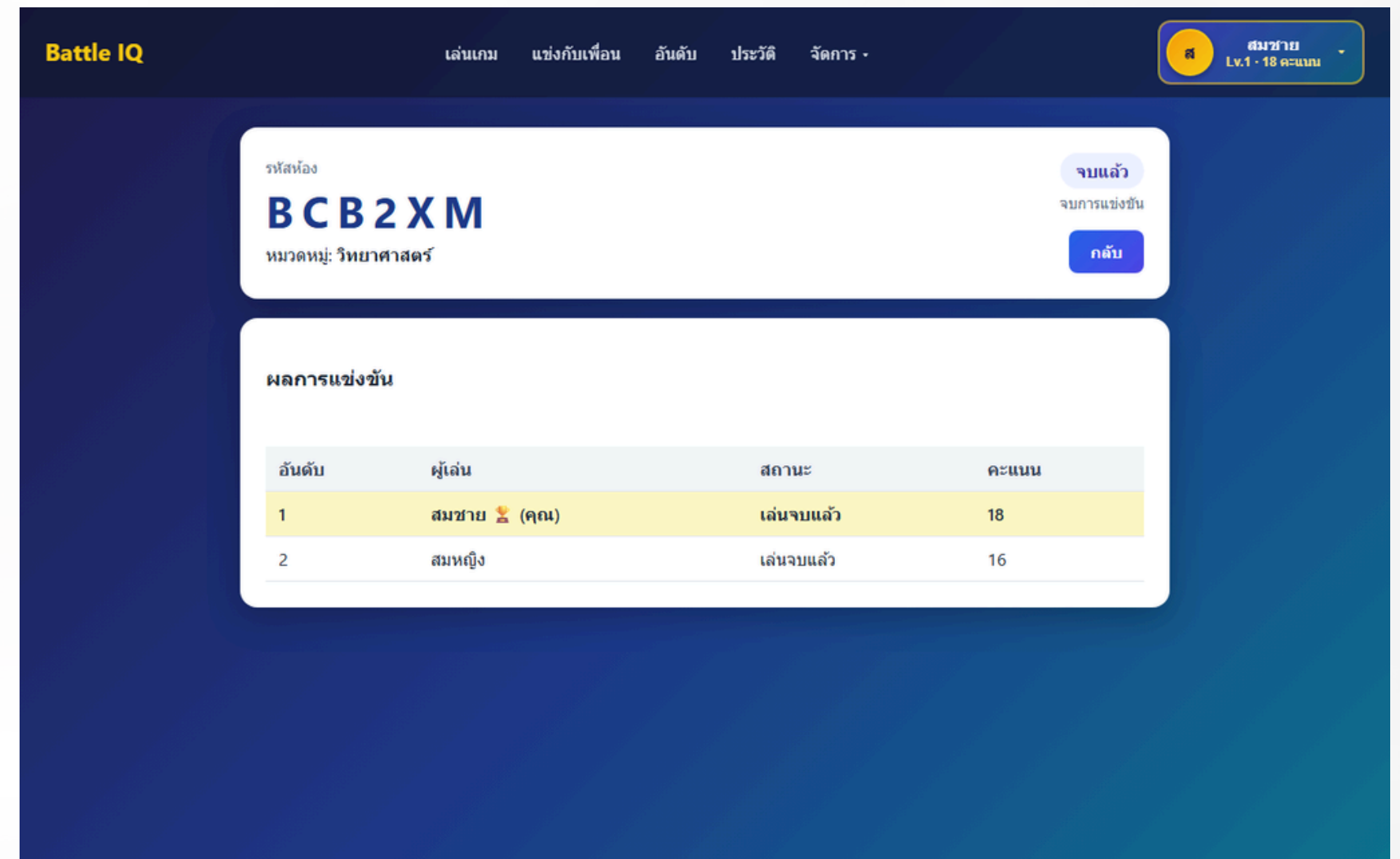
ข้อจำกัด

แนวทางพัฒนาต่อ

สิ่งที่ยังไม่ได้ทำในเวอร์ชันนี้ และสิ่งที่วางแผนไว้ถ้ามีเวลาต่อ

ข้อจำกัด: ห้องแข่งใช้ polling ทุก 2 วินาที ยังไม่ใช้ WebSocket, ยังไม่มีระบบลิมิตส์ผ่าน และ Render free tier จะกลับเมื่อไม่มีคนใช้

พัฒนาต่อ: เปลี่ยนเป็น WebSocket ให้แข่งแบบ real-time ระบบลิมิตส์ผ่าน และรองรับมือถือเต็มรูปแบบ



ขอบคุณครับ

Q & A

github.com/rattaphumk-tech/Battle-IQ



Battle IQ



battle-iq.onrender.com